

TravelMind

AI-Powered Travel Booking Assistant

Project Report

Eugene Martin

Student ID: 25105683

Individual Project

Table of Contents

1. Introduction	. 3
1.1 Project Overview	. 3
1.2 Code Repository	. 3
2. Background & Problem Setting	. 3
2.1 Problem Statement & Importance	. 3
2.2 Project Goals & Success Metrics	. 4
3. Analysis / Discussion	. 5
3.1 Software Platforms	. 5
3.2 Technology Architecture, Edge vs. Cloud	. 6
3.3 Data Requirements	. 6
3.4 Model Selection, Qwen3:8b	. 6
3.5 Model Exploration & Methodology	. 7
3.6 Data Privacy & Security	. 7
4. System Architecture	. 8
4.1 Architecture Diagram	. 8
4.2 Design Decisions	. 8
5. Client Architecture	. 10
5.1 Auth	. 10
5.2 Travel-Booking Components	. 10
5.3 Screens	. 11
6. Server Architecture	. 16
6.1 User Identity and Session Management	. 16
6.2 Controllers and Routes	. 16
6.3 Service Layer	. 17
6.4 Agent and Tools	. 17
6.5 Data Model	. 18
6.6 Database and Encryption	. 19
7. Testing Strategy	. 19
7.1 Unit Tests	. 19
7.2 End-to-End Tests	. 19
7.3 Other	. 20
8. Tool Calling and Performance Evaluation	. 20
8.1 JSON/Tool Calling Reliability	. 20
8.2 Performance Evaluation Report	. 20
9. Deployment	. 21
9.1 Docker Commands	. 22
10. Peer Reviews	. 23
10.1 Peer Review 1	. 23
10.2 Peer Review 2	. 23
11. Scalability and Future Work	. 24
11.1 Current Scalability Constraints	. 24
11.2 Proposed Scaling Work	. 24
11.3 Functional Future Work	. 24
References	. 24

1. Introduction

1.1 Project Overview

TravelMind is an AI-powered travel-booking assistant delivered as a web application. An Angular single-page client talks to an Express/TypeScript API server that wraps a locally-hosted Ollama large language model (Qwen3:8b) as a conversational “concierge” agent. A user describes what they need in plain language, a route and dates, a place to stay, a car to pick up, and the agent calls the Duffel API to retrieve live flight, hotel-stay, or car-hire offers and returns them as structured results in the chat. There is no booking form presented up front, the agent’s role is deliberately limited to search.

The application covers three travel verticals, **flights**, **stays**, and **cars**, which all follow the same basic pattern: i.e. the agent searches and the user books the chosen offering. Every chat session is tagged with exactly one of these three agent types when it is created i.e. each backed by its own system prompt, tool registry, and conversation history. A user assembles a multi-part trip (e.g. a flight plus a hotel) by holding several chats, one per vertical, under a single trip. This was a deliberate design decision, discussed further in section 3.5, that came from experimentation that was described in the original project proposal.

Booking itself is client-driven i.e. each search result card has a “Select this...” button that opens a passenger/guest/driver picker (depending on the vertical) in the UI. A logged-in user can build up a list of travel companions over time which can be used to book future trips. Flights go through a full Duffel payment-intent flow with an embedded, PCI-compliant payment widget while Stays and Cars complete immediately against pay-at-the-property / pay-at-pickup guarantee rates

The system runs almost entirely at the edge. Ollama and the model it serves run on the same machine as the application server, so no chat content is sent to an external AI provider. Duffel API is the only external network dependency. Confirmed bookings, chat history, user accounts, and saved traveller profiles are saved to a local, encrypted SQLite database, and a per-trip PDF itinerary can be generated and downloaded through the chat via a dedicated agent tool.

1.2 Code repository:

<https://github.com/martineug/travel-mind-app>

2. Background & Problem Setting

2.1 Problem Statement & Importance

Online travel booking is disjointed and demanding i.e. separate forms and tabs for flights, hotels, and car hire, each with its own filters for fare class, star rating, layovers, and pricing variation. TravelMind replaces those multi-step UI flows with a single conversational interface per vertical.

A user begins by describing his trip and can type something as loose as “cheap flights to Barcelona in June” or “an SUV in Rome next weekend,” and the agent resolves the relative dates, confirms the vertical search

parameters through a focussed question wizard to capture any missing criteria, then calls the appropriate Duffel search tool, and returns live, ranked offers inside the separate vertical chats.

A central aim of the project is to demonstrate that a locally-hosted, open-weight LLM can reliably drive real, live third-party APIs through native tool-calling, with zero dependency on an external AI API for the conversational layer. The only outbound network calls the system makes for AI purposes are to `localhost/host.docker.internal` and the only third-party API call it makes at all is to Duffel, purely for inventory and payment, never for language understanding.

The original proposal for this project assumed a single agent could be handed multiple tool domains (flight tools and hotel tools together) within one conversation and asked to keep track of dates, guest counts, and sequencing across both. In practice, the system that was built takes a different, more robust approach: one specialised agent per vertical, each with a narrower system prompt and a smaller, unambiguous tool registry, coordinated above the conversation level by grouping chats under a shared trip.

2.2 Project Goals & Success Metrics

The table below restates the goals set out in the original project proposal and records the current status of each against the delivered codebase.

• Goal (originally proposed)	• Current status
Complete a full flight booking (search → select → pay → confirmation) entirely through the chat interface, with no form-filling outside the payment widget	Delivered with question wizard. <code>flight_search</code> tool → passenger picker (exact adult count required by the offer) → POST <code>/flights/bookings/initiate</code> (Duffel payment intent) → <code><duffel-payments></code> widget → POST <code>/flights/bookings/confirm</code> → persisted to <code>flight_bookings</code> . A question wizard was added to capture search criteria for each vertical that are not captured in the initial chat
Complete a full hotel stay booking through the same conversational interface	Delivered, and extended to cars. <code>stay_search</code> / <code>car_search</code> tools → guest/driver picker → POST <code>/stays/bookings</code> or POST <code>/cars/bookings</code> (Duffel quote + book against guarantee-type rates) → persisted to <code>stay_bookings</code> / <code>car_bookings</code> . Car hire was listed as future work in the original proposal and has since been implemented as a full third vertical.
The AI correctly handles both flight and hotel bookings within a single trip-planning conversation	Replaced by a better design. Rather than one conversation spanning both tool domains, each chat is scoped to exactly one agent type(<code>flights</code> / <code>stays</code> / <code>cars</code>) A trip is planned by holding several chats, one per vertical, under one trip tab which avoids the cross-domain confusion the

• Goal (originally proposed)	• Current status
	original proposal flagged as a risk (see section 3.5) rather than attempting to solve it inside a single prompt.
The model automatically calls the correct Duffel tools with valid arguments, achieving a task completion rate above 80% across a fixed test suite	Done but not yet formally benchmarked. Every Ollama call and every tool call is recorded to a call_metrics table (success/failure, duration, token counts) via Tool.run()/recordOllamaMetric(), and a report generator turns that history into an HTML dashboard (section 8). A fixed-scenario completion-rate study, as originally scoped, has not yet been run against this, see Performance Evaluation.
End-to-end response latency under 20 s locally / 10 s on production hardware	Not yet measured against this target. The infrastructure to measure it (per-call duration_ms, p95/max, calls-per-minute) exists and is shown in section 8, but no latency study against these thresholds has been carried out.
Correctly parse relative date expressions (“next Friday”, “sometime in May”) into concrete YYYY-MM-DD values before calling the API	Delivered. No separate date-parsing tool exists. Instead, buildTodayContext() injects the real current date and weekday into every agent’s system prompt and the model resolves relative expressions itself before calling a search tool.

3. Analysis / Discussion

3.1 Software Platforms

The application is built on a TypeScript stack across all three packages in the repository (client/, server/, e2e/), each with their own dependencies:

- **Frontend:** Angular 21 [6], written completely with standalone components, providing the chat interface, trip tabs, and booking pickers.
- **Backend:** Express 5 [7] on Node.js (TypeScript), serving as the API layer between the frontend, the local LLM, and the Duffel booking APIs.
- **AI layer:** Ollama [2] running Qwen3:8b [1] locally, driven through the official ollama npm client using native function-calling.
- **Booking APIs:** Duffel Flights [3], Duffel Stays [4], and Duffel Cars [10] via @duffel/api, covering all three verticals. Payment capture for flights uses Duffel’s embedded @duffel/components payment widget.
- **Database:** SQLite using better-sqlite3-multiple-ciphers, encrypted at rest with SQLCipher [9].
- **Auth:** bcryptjs for password hashing and jsonwebtoken for session tokens.

- **PDF generation/parsing:** pdfkit for itinerary PDF generation and pdf-parse for the agent's pdf_extract tool.
- **Testing:** Vitest for both client and server unit tests, and Playwright [11] for the end-to-end package that creates a real headless browser against the real client and server.
- **Logging:** pino/pino-http logging on the server, plus a custom call_metrics table and HTML report generator (section 8).
- **Deployment:** Docker Compose, with an nginx container serving the built Angular app and reverse-proxying /api/*, and a node container running the compiled Express server.

3.2 Technology Architecture, Edge vs. Cloud

The AI component runs entirely at the edge. Ollama hosts Qwen3:8b on the same machine as the application server (or, in the Docker deployment, on the host machine, reached from the server container via host.docker.internal, Ollama is deliberately not containerised, to avoid re-downloading a multi-gigabyte model into an image). No conversational data, including passenger names, dates of birth, or trip details discussed during search, is ever sent to an external AI provider eliminating per-token API cost.

Duffel is the only cloud dependency, and it is scoped narrowly: live flight/stay/car inventory search and, for flights, payment-intent creation and order confirmation. Where the project's sandbox Duffel key does not yet have Stays or Cars access enabled, USE MOCK STAYS/USE MOCK CARS flags swap in local result providers. USE MOCK FLIGHTS setting does the same for flight results if a complete local environment is required during testing.

3.3 Data Requirements

Flight, stay, and car inventory is retrieved live from Duffel on every search or from mock providers if required, there is no offline dataset to curate or pre-process, and no annotated dataset was required to build the agent.

Qwen3:8b instruction-following and native tool-calling ability are used as found. The “annotation” layer is the system prompt and tool schema for each of the three agent types (AGENT_SYSTEM_PROMPTS in prompts.ts), each tells the model what persona to adopt, the exact JSON contract to respond with ({"action": "chat" | "search", ...}), and which search tool it has access to.

User-provided input is natural-language chat messages, plus the structured details collected once at registration (name, email, phone, date of birth, gender, title) and reused as the account holder's identity on every booking, and any additional travellers saved via the passenger picker. Search results returned by Duffel are sliced by each search tool to keep the payload within the model's context window and the UI readable, rather than passed through in full.

3.4 Model Selection, Qwen3:8b

The project uses Qwen3:8b [1], an open-weight 8-billion-parameter model from Alibaba's Qwen team, served locally via Ollama. It was selected for its native tool-calling support, extended-reasoning (“thinking”) mode, and its ability to run efficiently on Apple Silicon without a discrete GPU.

Qwen3:8b is a general-purpose model repurposed into a travel concierge purely through prompt engineering and agentic tool design, no fine-tuning was performed. NativeToolAgent is constructed with a system prompt string and a ToolRegistry and simply runs Ollama's native function-calling loop until the model returns a plain-text answer instead of a tool call.

All three verticals (flights/stays/cars) run through the same agent implementation, configured differently with their own system prompt.

3.5 Model Exploration & Methodology

Agentic tool use. Each agent's tool registry is scoped to exactly one search tool (flight_search, stay_search, or car_search) plus a shared set of general-purpose tools (calculator, file read/write/list, save-memory, PDF extract, itinerary PDF generation). The model decides when to call its search tool and iterates (up to a configurable maxIterations, default 8) until it returns a final JSON answer instead of another tool call.

One agent per conversation, each scoped to a single vertical. The original proposal's hardest identified challenge was "the model must correctly sequence flight and hotel tool calls within the same conversation without mixing up dates or guest counts." Rather than solving that inside a single, larger prompt, the architecture sidesteps it so that each chat session is tagged with one agentType at creation, so its tool registry only ever holds one vertical's search tool, and a trip is planned across multiple chats instead.

Structured output / JSON mode. Every system prompt enforces a strict contract: the model must respond with only raw JSON matching either {"action":"chat","message":"..."} or {"action":"search","message":"...","flights|stays|cars":[...]}. Because the model has been observed dropping the results array or inventing figures, applyAuthoritativeToolResults in chatbot-service.ts always overwrites whatever the model claims to have found with what the tool call actually returned.

Extended thinking. OLLAMA_ENABLE_THINKING toggles Qwen3:8b's thinking mode via the think parameter on ollama.chat(). The call_metrics table already records enough (duration, token counts, success) to compare thinking on vs. off once a fixed evaluation suite is run against it (section 8).

Context window management. NativeToolAgent.isNearLimit() estimates token usage from character counts against the configured num_ctx and, once past an 80% threshold, calls summariseHistory() to compress everything except the most recent messages into a single summary turn, persisted per chat via ChatSummaryRepository so it survives a server reload.

Iteration limit. OLLAMA_MAX_ITERATIONS is a runtime-configurable cap on the agentic loop, defaulting to 8 in development. If the model still hasn't produced a final answer by then, the loop returns a "reached step limit" message rather than looping forever.

3.6 Data Privacy & Security

The full SQLite database file is encrypted at rest via SQLCipher (through better-sqlite3-multiple-ciphers), keyed with a single DB_ENCRYPTION_KEY set via the cipher/key settings when the database connection is opened (server/src/db/index.ts).

Passwords are never stored, only a bcrypt hash in `user_profiles.password_hash`. Sessions are JWTs signed with `JWT_SECRET` which are valid for 7 days, delivered as `httpOnly` so the token is inaccessible to client-side JavaScript. `requireUser()` (in `session.ts`) is the single shared guard every authenticated route calls to resolve the signed-in user or rejects it with a 401 error.

Passenger/guest/driver details entered in a picker are sent directly from the client to the server's booking endpoints and from there to Duffel over HTTPS, they are never routed through the LLM, since the agent has no booking tools and never sees this data. Card payment data for flights is captured entirely inside Duffel's own PCI-compliant payments component.

The Duffel API key itself is a server-side environment variable, never exposed to the client bundle. Because Ollama runs locally, the natural-language content of every conversation, including anything a user types about their trip, never leaves the machine running the server, which remains the project's core privacy guarantee.

4. System Architecture

4.1 Architecture Diagram

The two figures below trace a request from the browser through to the two external systems the application depends on, the local Ollama instance and the Duffel API, split at the point where the service layer hands off into the agent layer.

Figure 1 covers the client, the nginx layer (Docker deployment only), and the Express controllers/session guard/service layer:

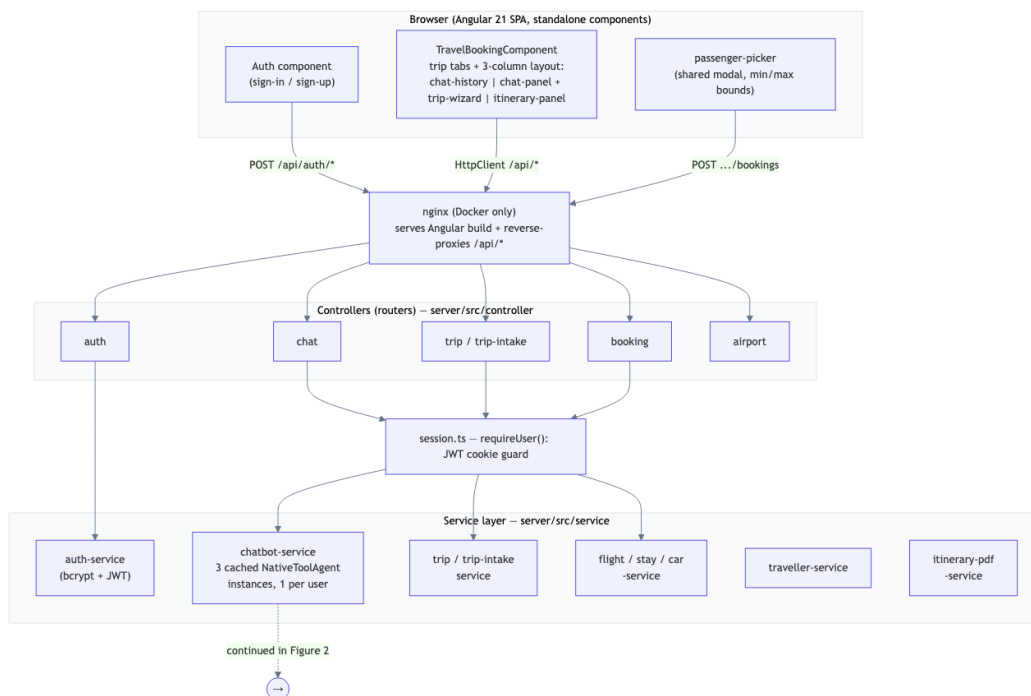


Figure 1, client, routing, and service layer

Figure 2 picks up from chatbot-service and covers the agent layer, its tools, the repositories, the encrypted database, and the two external systems:

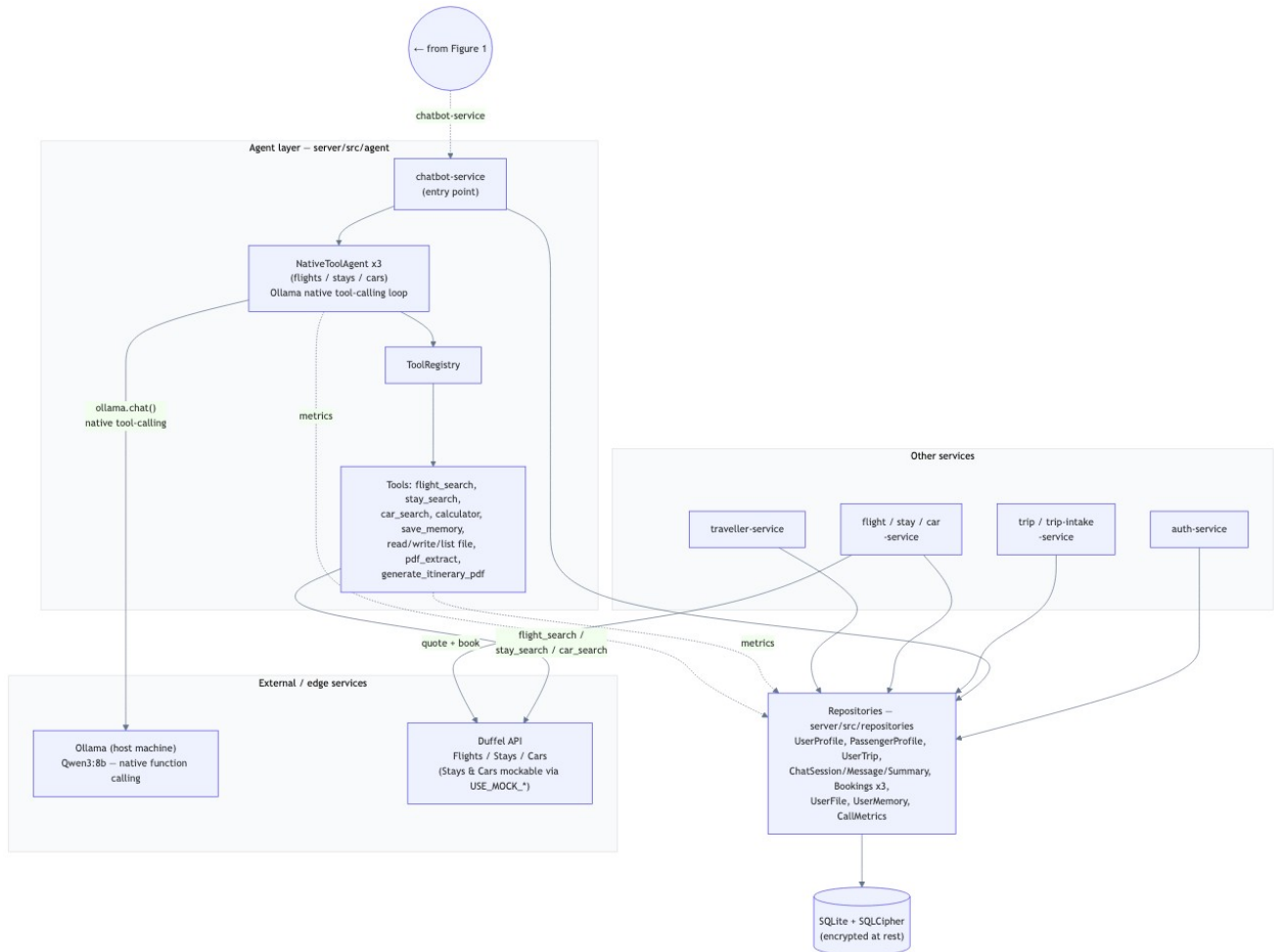


Figure 2, agent, data, and external services

4.2 Design Decisions

Three important architectural decisions:

- **The agent never books.** Every arrow into Duffel from the *tool* layer is a search call i.e. the only arrows that reach Duffel for booking/payment originate from the flight/stay/car services, which are invoked directly by booking-controller from the client's person-picker submission, never through the LLM.
- **One agent instance per chat, cached for the chat's lifetime.** chatbot-service lazily builds a NativeToolAgent the first time a chat is accessed and caches it keyed by chat ID, so repeated turns in the same conversation reuse the same instance rather than reconstructing one per request. Idle entries are evicted after a TTL(Time To Live).

- **Separate agentType per vertical chat.** From a UX standpoint, tying an agentType permanently to the chat session, rather than letting one chat span flights, stays, and cars, trades conversational continuity for user focus. The main advantage is that each chat stays focussed and readable and the greeting, suggestion chips, and placeholder text in chat-panel-config.ts relate directly to "search flights" or "search stays" etc. instead of trying to satisfy all three. Also the pill-list history with vertical-type colours reinforces this separation at a glance. The downside is that for the common case of planning one trip end-to-end i.e. a user booking a flight, a hotel, and a rental car for the same trip has to explicitly start three separate conversations but there is an attempt to automate this from the trip planning screen at the beginning.

5. Client Architecture

5.1 Auth

client/src/app/auth/auth.component.ts renders a single sign-in/sign-up screen. AuthService (core/auth.service.ts) wraps HttpClient calls to /api/auth/sign-up, /sign-in, /sign-out, and /me, and exposes the resolved user as a currentUser read by top-banner.component.ts and the chat UI.

The route guard authGuard (core/auth.guard.ts) calls GET /api/auth/me on app load to decide whether the root route (TravelBookingComponent) or the /auth route is shown.

5.2 Travel-Booking Components

client/src/app/travel-booking/ is the main feature area, structured as a set of standalone components composed under travel-booking.component.ts:

- **chat-history/**, the conversation pill list for the active trip: drag-to-reorder via @angular/cdk/drag-drop, a colour accent per agent type, delete, and an anchored "new conversation" dropdown for picking flights/stays/cars for the next chat.
- **chat-panel/**, the message thread and input box. chat-panel-config.ts centralises all per-agent-type settings such as header title, greeting, suggestion chips, and the input placeholder.
- **trip-wizard/** and **wizard-questions/**, a short guided-question flow (trip-intake-controller) used to gather a new trip's basics before handing off to each of the vertical agents.
- **passenger-picker/**, one component reused for all three booking types, parametrised by min/max selection bounds i.e. exactly the offer's adult count for flights, 1->(adult count) for stay guests, and exactly 1 for a car's main driver. People are sourced from GET /api/passengers (the account holder, plus everything saved in passenger_profiles). Anyone newly added is updated/inserted back into passenger_profiles on submit so they can be reused on a later booking.
- **itinerary-panel/**, the third column of the layout, listing confirmed flight/stay/car bookings for the active trip (GET /trips/:tripId/bookings/{flights, stays, cars}) and exposing the generated itinerary PDF.
- **agent-icon/**, a single component rendering one of three SVG line icons (flight/bed/car) by AgentType, reused by both the chat pills and the new-conversation dropdown.

5.3 Screens

The client has exactly two top-level routes (app.routes.ts): /auth (sign-in/sign-up) and / (the main app, guarded by authGuard). Within the main route, TravelBookingComponent renders:

1. **Top banner**, app logo and the signed-in user's menu (sign-out).
2. **Trip tabs**, tabs across the top, one per UserTrip, each closable with a per-tab X.
3. **A resizable three-column layout** for the active trip: chat history (left) → chat panel, including the trip-wizard when a new trip needs its basics gathered (centre) → itinerary panel (right).
4. **Passenger-picker modal**, opened on top of this layout from a search-result card's "Select this..." button, for whichever vertical the active chat is booking.

•

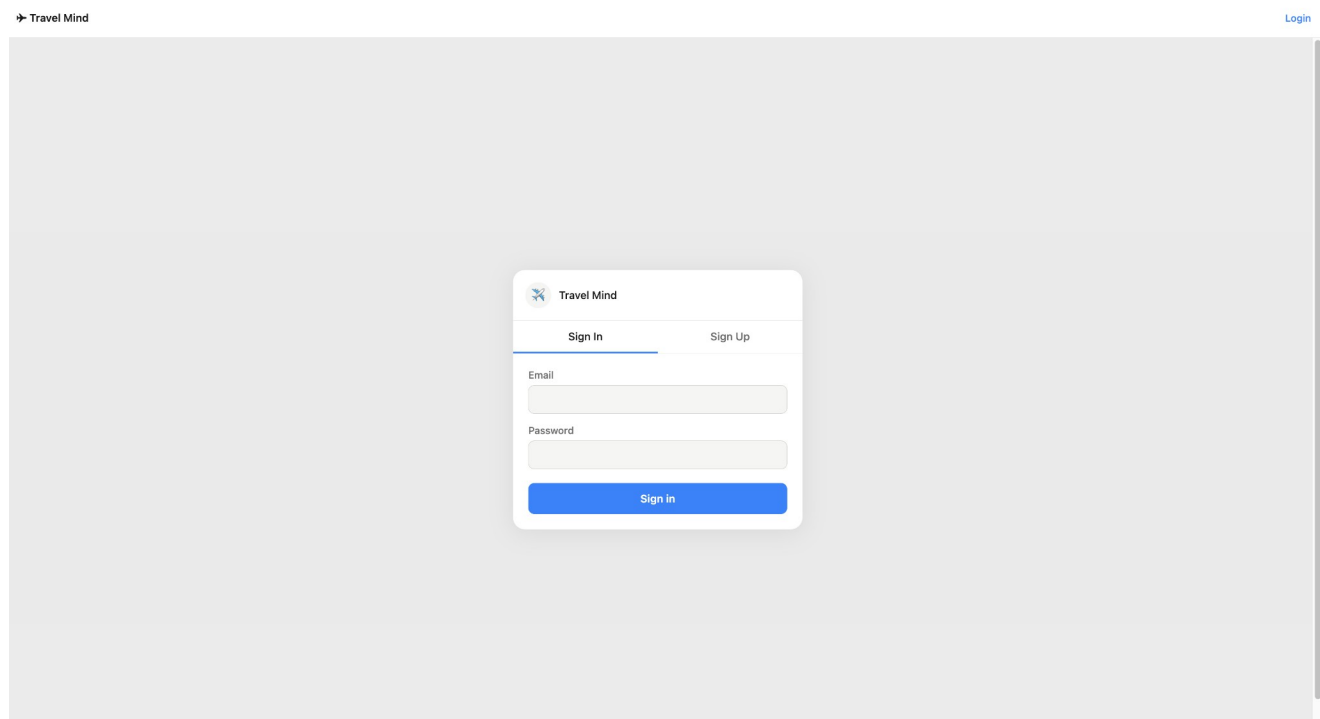
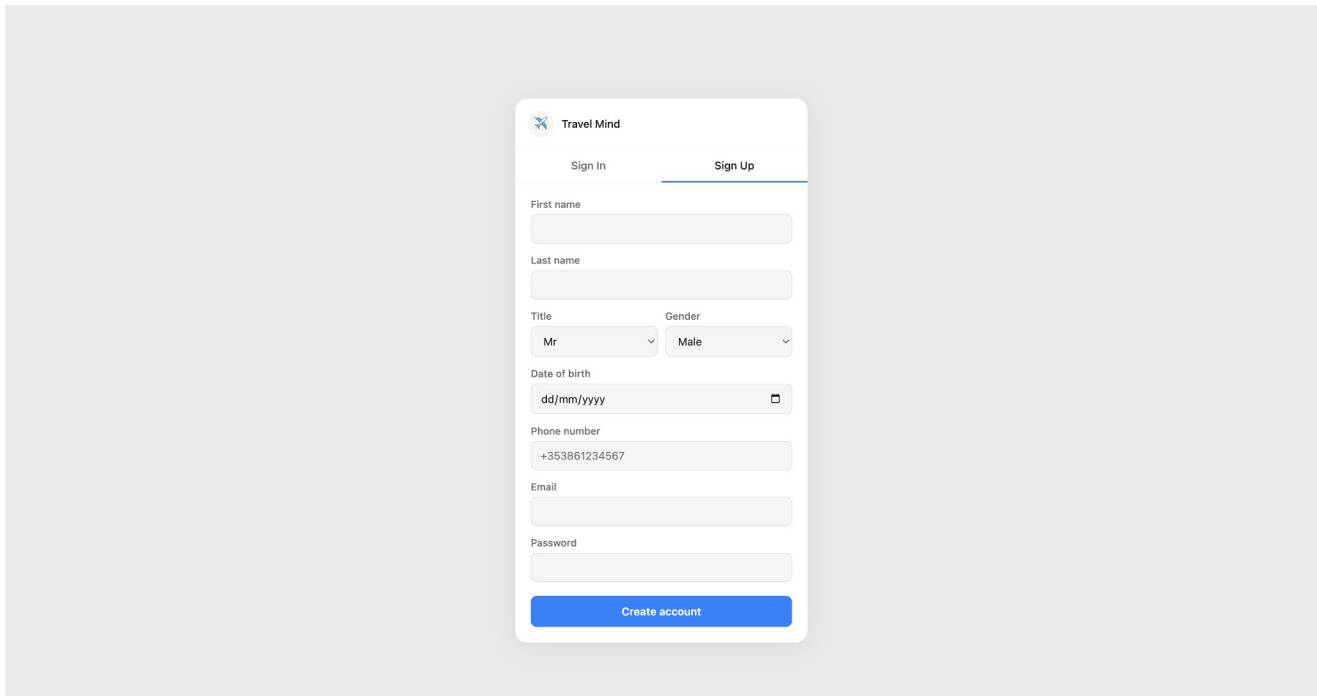
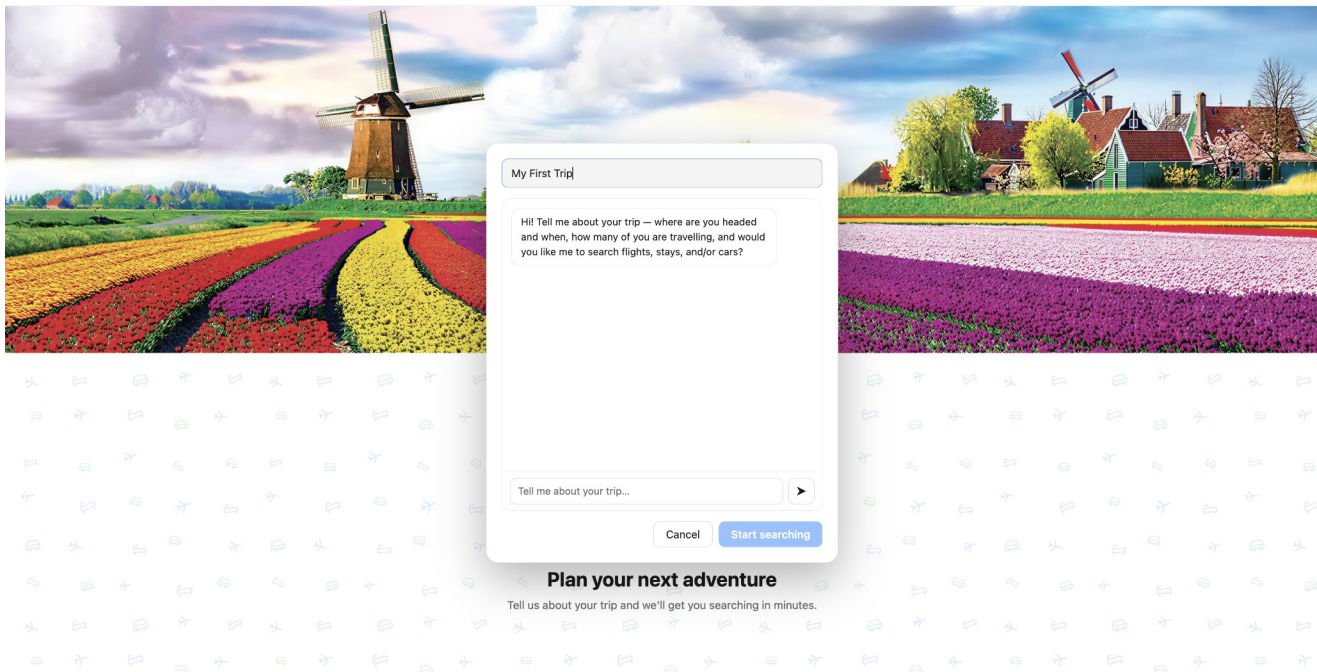


Figure 3, Sign in screen



The sign up screen features a white modal form centered on a light gray background. At the top left of the form is the Travel Mind logo. Below it are two tabs: 'Sign In' and 'Sign Up', with 'Sign Up' being the active tab. The form contains several input fields: 'First name', 'Last name', 'Title' (a dropdown menu with 'Mr' selected), 'Gender' (a dropdown menu with 'Male' selected), 'Date of birth' (a date picker showing 'dd/mm/yyyy'), 'Phone number' (pre-filled with '+353861234567'), 'Email', and 'Password'. At the bottom of the form is a blue button labeled 'Create account'.

Figure 4, sign up screen



The trip plan chat screen features a white chat modal centered on a background image of a Dutch landscape with a windmill and tulip fields. The modal has a title bar 'My First Trip' and a text input area with the placeholder 'Tell me about your trip...'. Below the input area is a blue button labeled 'Start searching' and a 'Cancel' button. The background image is framed by a pattern of small, colorful travel-related icons (airplane, car, train, etc.) at the bottom.

Figure 5, trip plan chat screen

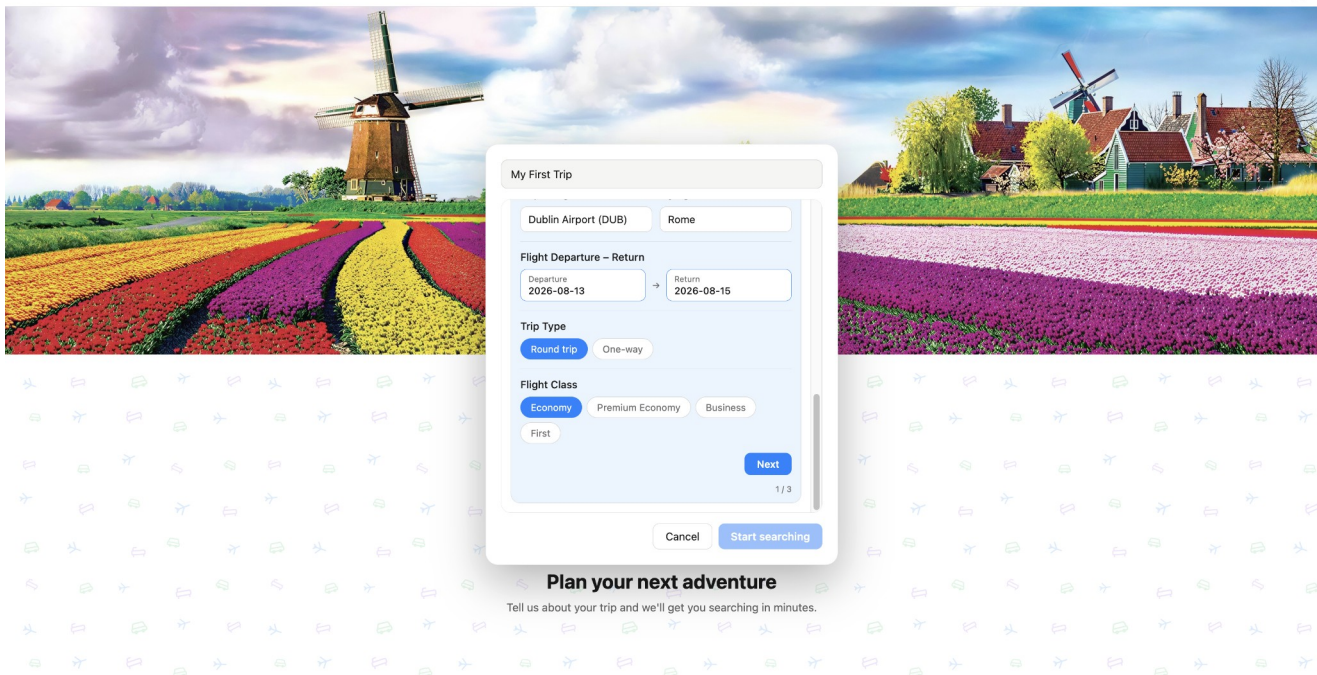


Figure 6, trip wizard questions screen

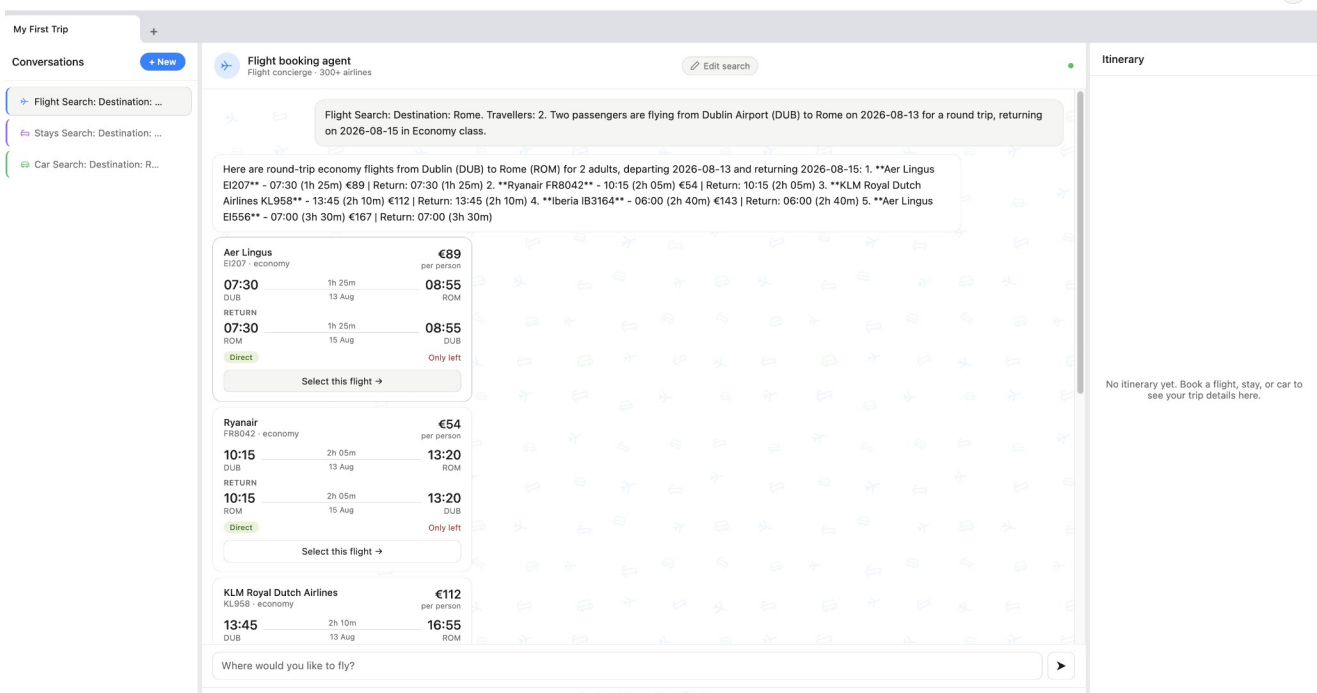


Figure 7, trip tabs, chat tabs and flight search results screen

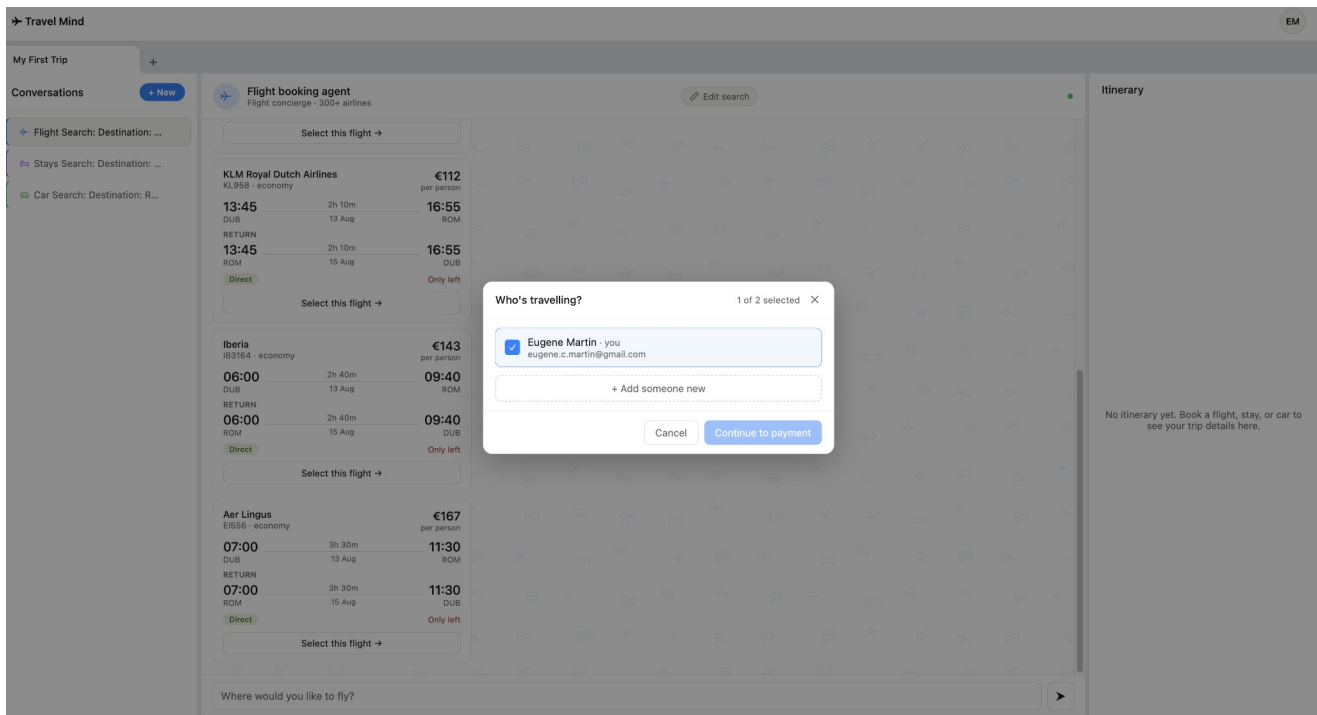


Figure 8, passenger picker screen

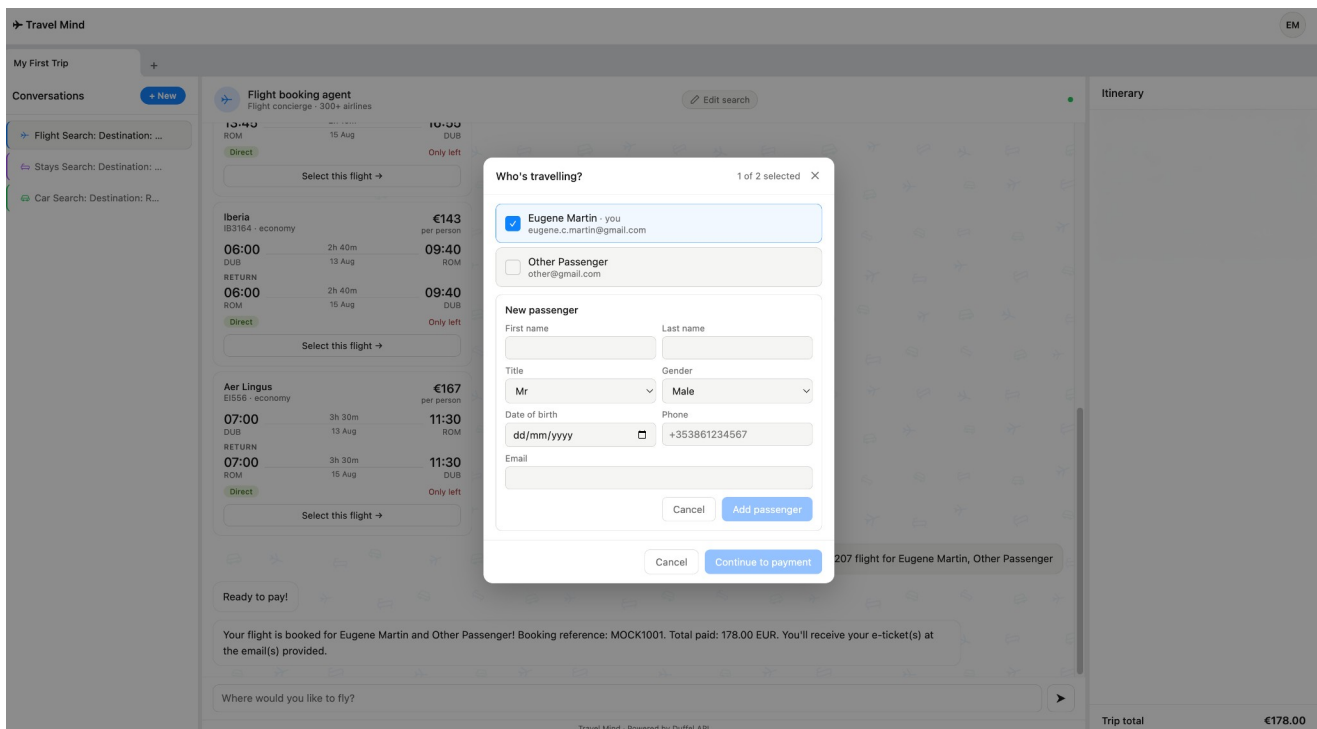


Figure 9, new passenger screen

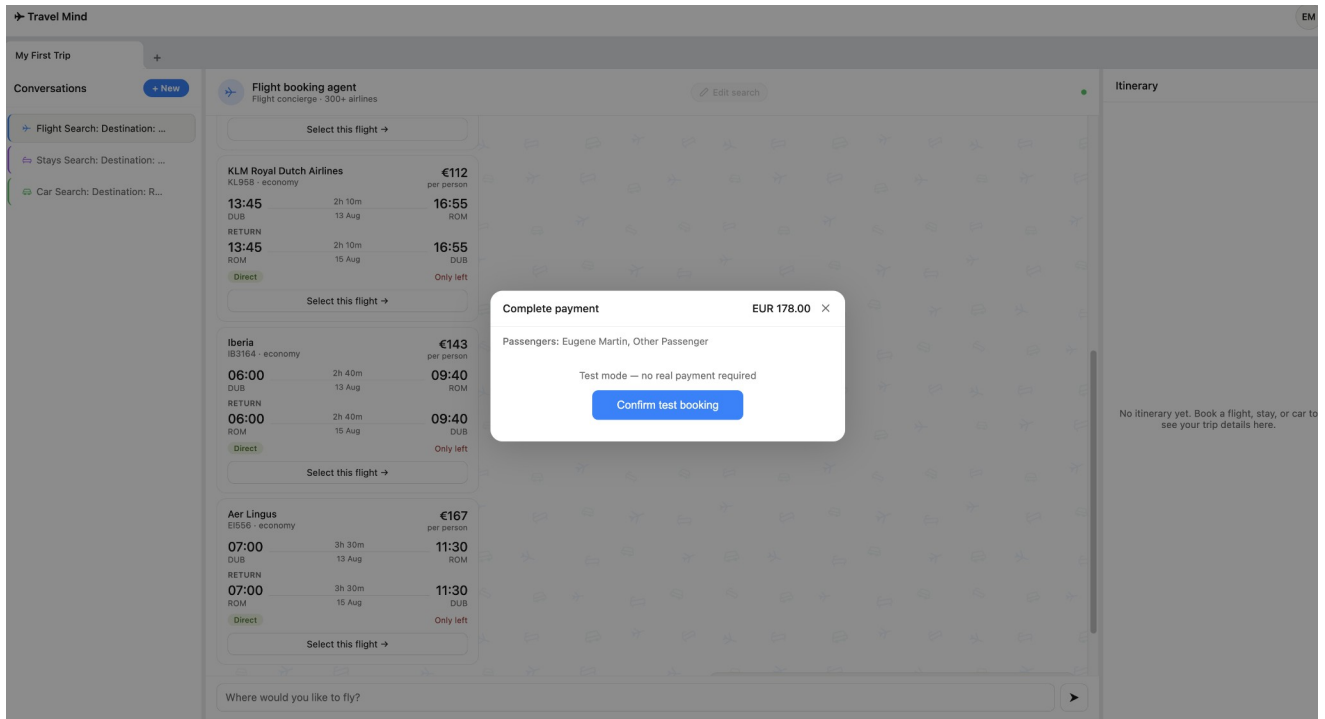


Figure 10, mock payment screen

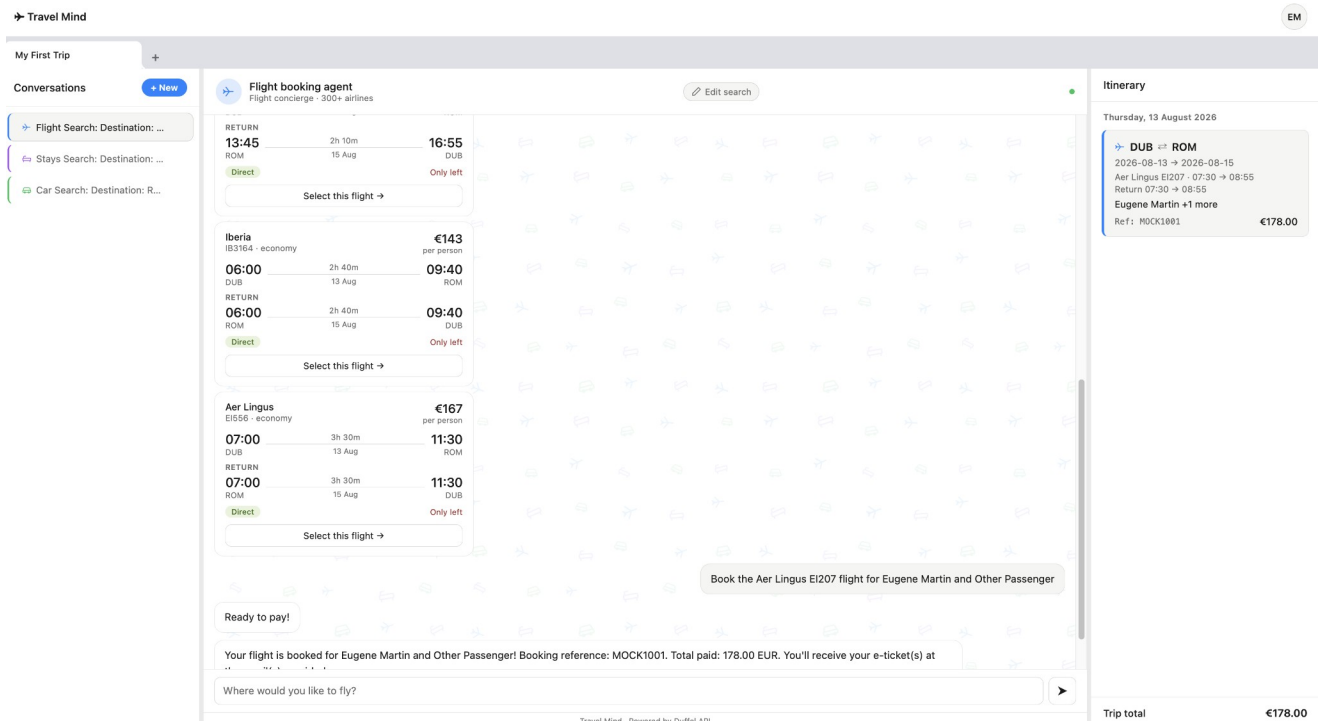


Figure 11, itinerary screen

6. Server Architecture

6.1 User Identity and Session Management

user_profiles is the single source of truth for the account holder storing hashed credentials alongside their own traveller identity i.e. phone_number, born_on, gender and title captured once at sign-up.

auth-service.ts handles sign-up creating a default “My First Trip” for the new user and sign-in, and issues a jsonwebtoken containing only { userId }, signed with JWT_SECRET with a 7-day expiry. auth-controller.ts sets this token as an httpOnly session_token.

Every other controller that needs the signed-in user calls the shared requireUser(req, res) helper in session.ts, which reads the cookie, verifies it via getUserFromToken, and, on failure, writes the 401 response itself and returns null.

6.2 Controllers and Routes

All routes are mounted under /api in index.ts. Six Express routers cover the whole API:

Router	Representative routes
auth-controller	POST /auth/sign-up, /auth/sign-in, /auth/sign-out, GET /auth/me
chat-controller	GET /chatbot PUT /chatbot/current-trip POST /chatbot/chats GET/DELETE /chatbot/chats/:chatId PATCH /chatbot/chats (reorder) GET/POST /chatbot/chats/:chatId/edit-search POST /chatbot/chats/:chatId/cancel-payment POST /chatbot/message POST /chatbot/files GET /chatbot/files/:filename
trip-controller	POST/PATCH/DELETE /trips[:tripId], GET /trips/:tripId/bookings/{flights, stays, cars}
trip-intake-controller	POST /trip-intake/{message, summary, vertical-questions}
booking-controller	POST /flights/bookings/{initiate, confirm}, POST /stays/bookings, POST /cars/bookings

Router	Representative routes
	GET /passengers
airport-controller	GET /airports/suggest

6.3 Service Layer

Controllers are thin, delegating to a service per concern:

- **auth-service**, sign-up/sign-in, password hashing, JWT issuance/verification.
- **chatbot-service**, one instance per logged-in user (cached in a Map in chat-controller.ts), lazily building one NativeToolAgent per chat on first access and caching it by chat ID.
- **trip-service** / **trip-intake-service**, trip CRUD, and the short guided-question flow used to start a new trip.
- **flight-service** / **stay-service** / **car-service**, the booking logic per vertical. Duffel payment-intent + order confirmation for flights; Duffel quote + booking for stays and cars.
- **traveller-service**, saveTravellers, update/inserting anyone chosen in a passenger picker into passenger_profiles.
- **itinerary-pdf-service**, renders a trip's confirmed bookings into a downloadable PDF using pdfkit, invoked through the agent's generate_itinerary_pdf tool.
- **memory-service** / **file-service**, back the agent's general-purpose save_memory and file read/write/list tools respectively.

6.4 Agent and Tools

NativeToolAgent (agent.ts) is the whole agentic loop, and is completely domain-agnostic, it knows nothing about flights, stays, or cars. It is constructed with a systemPrompt and a ToolRegistry, and its run(query) method repeatedly calls ollama.chat executing whatever tools the model calls and passing the result back as a role: 'tool' message, until the model responds without a tool call or maxIterations is reached.

Every iteration is recorded to an AgentTrace[] (tool name, arguments, truncated and full result) for later debugging, and every Ollama call and tool call is separately timed and written to the call_metrics table.

ToolRegistry is a simple map of Tool instances. Tool.toOllamaSchema() turns each one into the JSON-schema that Ollama's native tool-calling expects. chatbot-service.ts builds three registries per user, one per agent type, grouping a domain tool (flight_search, stay_search, or car_search) on top of a shared general registry available to all three i.e. calculator, read_file/write_file/list_files, save_memory, pdf_extract, and generate_itinerary_pdf.

flight_search/stay_search/car_search each wrap the corresponding @duffel/api search call, flatten the response, and slice it down to a small, fixed number of ranked results before returning, keeping the payload

inside the model's context budget. Stays and Cars take latitude/longitude directly rather than a place name which the model supplies coordinates from its own knowledge

6.5 Data Model

The table below reads top-down as parent to child.

Entity	Table(s)	Notes
User	user_profiles	Credentials + the account holder's traveller identity
› PassengerProfile	passenger_profiles	Other saved travellers, offered in booking pickers
› UserTrip	user_trips	Pure grouping container, just a name
› › ChatSession	chat_sessions	id, tripId, agentType, sort_order
› › › ChatMessage	chat_messages	role/content
› › ChatSummary	chat_summaries	LLM-generated summary per chat
› › Bookings	flight_bookings / stay_bookings / car_bookings	One row per booking, keyed by trip_id

These back general-purpose agent features and are not part of the trip hierarchy above, they hang off user_id directly rather than a trip.

Table	Backs	Notes
user_memories	The agent's save_memory tool	Notes the agent has saved about a user
user_files	The agent's file read/write/list tools	Files a user has uploaded or the agent has generated
call_metrics	Performance monitoring (see Performance Evaluation)	Populated by the agent and tool layer itself.

6.6 Database and Encryption

The database is SQLite accessed through better-sqlite3-multiple-ciphers, which bundles SQLCipher. On first connection (server/src/db/index.ts), the driver sets cipher='sqlcipher' and key='<DB_ENCRYPTION_KEY>' so the entire .db file on disk is encrypted at rest.

7. Testing Strategy

7.1 Unit Tests

Both client and server use Vitest for testing.

On the server unit tests cover the following:

Agent layer - parse-agent-json.test.ts, tool.test.ts, vertical-questions.test.ts, ollama-gate.test.ts, tools/itinerary-pdf.test.ts

Model - agent-type.test.ts

Utilities - time-date.test.ts

Providers - duffel-retry.test.ts

Repositories - chat-message-repository.test.ts

Service layer - auth-service.test.ts, car-service.test.ts, flight-service.test.ts, stay-service.test.ts, traveller-service.test.ts, trip-service.test.ts, trip-intake-service.test.ts, itinerary-pdf-service.test.ts, chatbot-service.test.ts

On the client, unit tests cover the following:

Utilities - parse-agent-json.spec.ts, format-wizard-answers.spec.ts

Component specs - app.component.spec.ts, chat-history.component.spec.ts, itinerary-panel.component.spec.ts, passenger-picker.component.spec.ts

7.2 End-to-End Tests

A separate e2e/ package uses Playwright to drive a real Chromium browser against the real client and real server (not mocks of either), configured with NODE_ENV=e2e against an isolated database at server/data/e2e.db so a test run never touches the developer's own data.

Four spec files cover the full user journey:

auth.spec.ts (sign-up/sign-in/sign-out, wrong-password handling),
trips.spec.ts (trip creation/rename/delete),
trip-wizard-and-flights.spec.ts (the guided intake flow through to a flight search and booking), and
stays-and-cars.spec.ts (search → pick → book for both remaining verticals). Duffel itself stays mocked

7.3 Other

- **Agent trace logging.** NativeToolAgent.run() returns a full AgentTrace[] alongside every answer, every tool called, the arguments passed.
- **Call-metrics recording.** Independent of the trace above, every Ollama call and every tool call is durably recorded to the call_metrics table

8. Tool Calling and Performance Evaluation

8.1 JSON / Tool-Calling Reliability in Practice

Ollama's native tool-calling proved reliable for triggering the correct search tool with well-formed arguments, but the model's own JSON output was less trustworthy. It was observed dropping search-result arrays it was told to copy verbatim through the system prompt, so applyAuthoritativeSearch overwrites the model's claimed results with what the tool actually returned rather than trusting the model's transcription. An earlier design iteration let the model handle booking/payment directly via a flight_book tool, which exposed a more serious failure i.e. the model inverting payment tokens. Since then, booking was redesigned to bypass the model entirely and the flight_book tool has been removed from the codebase

8.2 Performance Evaluation Report

Every call the system makes to Ollama (the main tool-calling loop, and history-summarisation calls) and every tool invocation is recorded to the call_metrics table as it happens.

Each row captures: call type (ollama_chat / ollama_summarize / tool_call), the model or tool name, an optional label (which agent/vertical it belongs to), success/failure and error text, duration in milliseconds, prompt/completion token counts and load/prompt-eval/eval timing, and the truncated request/response payload

npm run report:performance (can be narrowed with --since=24h/--since=7d) turns that history into a static HTML dashboard, located in server/reports/performance-report.html, covering:

- Summary cards: total calls, success rate, average/p95/max duration.
- Calls-per-minute and average-processing-time-per-minute time series (Chart.js).
- A breakdown table per call type/name/label, including average prompt/completion tokens and tokens/sec for Ollama calls.
- Recent calls and recent failures, each with its full (truncated) request/response payload

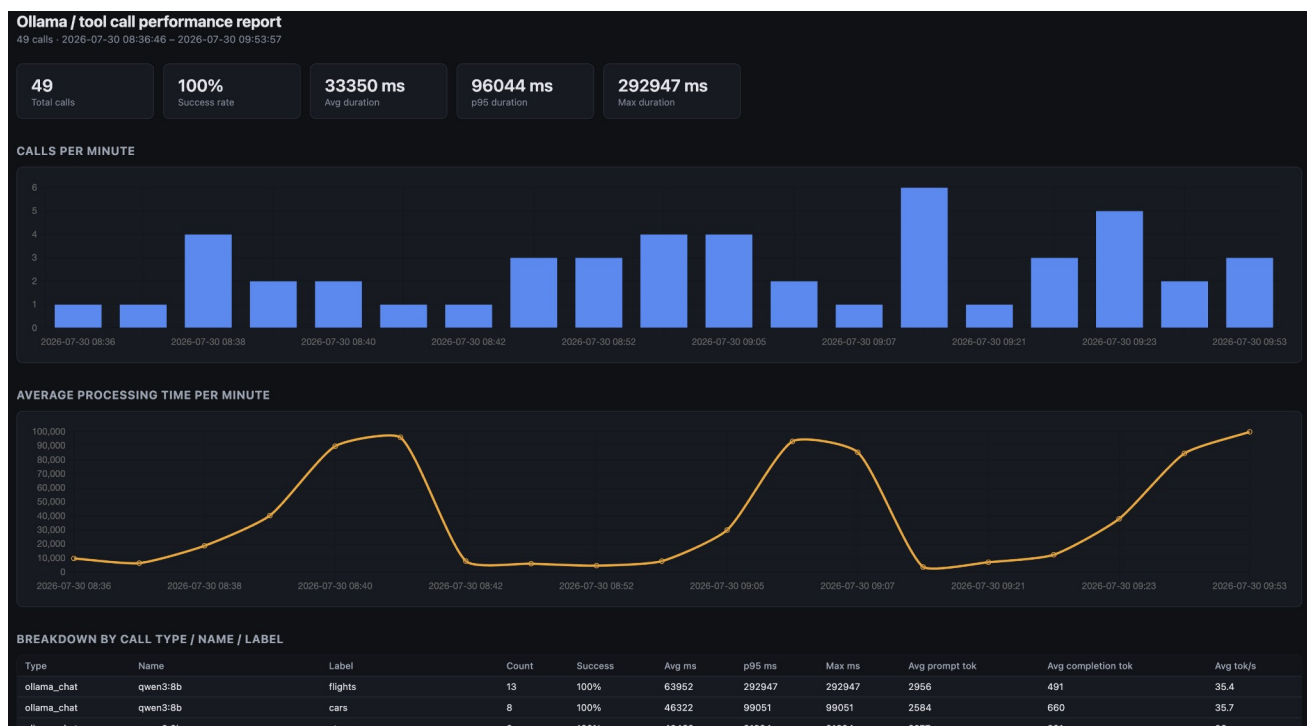


Figure 12, performance report

9. Deployment

Two containers, orchestrated by a single docker-compose.yml at the repo root: an nginx container serving the built Angular client and reverse-proxying /api/* to the server (mirroring what client/proxy.conf.json does in development), and a node container running the compiled Express server.

Ollama itself is intentionally not containerised, the server container reaches it on the host via host.docker.internal, configured through the OLLAMA_HOST environment variable. Database data is stored in server/data/ on the host

9.1 Docker Commands

Build both images and start the stack (nginx on :80, server behind it)

`docker compose up --build`

Rebuild after a code change

`docker compose up --build`

Tear down (containers only, server/data/ on the host is untouched)

`docker compose down`

Follow logs for the server container

`docker compose logs -f server`

Prerequisites for the Docker Deployment

- Docker with **Compose v2** (docker compose, not the standalone docker-compose binary).
- **Ollama running on the host** with the configured model already pulled (ollama serve + ollama pull qwen3:8b) - Docker does not start or manage Ollama.

To install Ollama:

```
brew install ollama # or see ollama.com for other platforms
```

```
ollama serve & ollama pull qwen3:8b # must match OLLAMA_MODEL  
in server/env/.env.local
```

- If the Duffel key in use does not have Stays/Cars access enabled, **USE MOCK STAYS/USE MOCK CARS** added under the server service's environment: block. They are deliberately set to true in server/env/.env.production. change them to false there, or override via the server service's environment: block in docker-compose.yml, once real Duffel access is available

Runtime Notes

- Once running, the application is reached at **http://localhost** - not port 3000, which is only the server container's internal port, reverse-proxied by nginx.
- There is **no named Docker volume** in this setup (docker compose down -v has nothing to remove).

10. Peer Reviews

Peer Review 1:

Suggestions for Improvement	Action
Include more information on code design decisions and the rationale behind key implementation choices.	Done. See Design Decisions in Section 4.
Consider providing additional details on error handling and exception management.	TODO
Ensure coding standards are consistently followed across all modules to improve maintainability.	Coding standards are consistently followed
Add an architecture diagram to help readers quickly understand the overall system.	This already exists in the report
Include screenshots or sample outputs to improve the user experience.	Done. Added UI screenshots
Move advanced setup and troubleshooting information into separate documents to make the README easier to navigate.	Done. Extracted QUICKSTART.md from README.md
Explain key design decisions and architectural trade-offs in more detail.	Done. See Design Decisions in Section 4.
Include a discussion of scalability, security, and future enhancements.	Done. See Scalability and Future Work in Section 11

Peer Review 2:

Suggestions for Improvement	Action
Add a short quick-start guide for reviewers.	Added QUICKSTART.md document
Include screenshots for people who can't get Ollama or Duffel working locally.	Added UI screenshots
Add some real benchmark or evaluation results (response times, success rates, failure cases).	Added performance report screenshot. See Section 8.2
Explain more about how reliable the JSON/tool-calling setup is in practice.	Added JSON/Tool Calling and Reliability . See Section 8.1
Discuss the one-chat-per-vertical design trade-off more from a user-experience perspective.	Done. See 'Separate agentType per vertical chat' in Section 4.
Fix the GitHub repository link, which returned a 404 Not Found error.	This is not an error. It will always return 404 when you don't have access. Will change it to public access
Investigate the bug where entering a flight destination in the UI produced no result.	This has been addressed

11. Scalability and Future Work

Current scalability constraints:

Session/agent state is held in-process - ChatBotService caches one NativeToolAgent per active chat in an in-memory Map, so it wouldn't survive a restart and can't be shared across multiple servers

SQLite performance - SQLite is best suited for a single-writer, single-host deployment but would bottleneck under concurrent writes from multiple server instances.

Ollama concurrency - Ollama runs as a single local process on the host, so every chat request across all three verticals competes for one inference queue with no horizontal scaling path.

Proposed scaling work:

Externalise agent/session state to a shared store so any server instance can resume any user's session, enabling stateless horizontal scaling behind a load balancer.

Migrate from SQLite to a networked RDBMS to remove the single-writer limitation.

Introduce a proper schema-migration framework, replacing the current dev-only CREATE TABLE IF NOT EXISTS-and-drop approach, as a prerequisite for evolving the schema against live data.

Scale inference from a single local Ollama process to a pool of workers

Functional future work:

Enable real Duffel access for Stays and Cars once account-level block is lifted, removing reliance on the fixture-backed mock providers.

Extend the client-driven booking pattern to support payment collection for Stays/Cars which is currently limited to pay-at-property/pay-at-pickup rate types.

Extend history summarisation to support longer-running, multi-day trip-planning sessions without losing earlier context.

Expand the unit and e2e test suites with load testing to validate behaviour at the concurrency levels that a multi-instance deployment would introduce.

References

[1] Qwen Team, "Qwen3 Technical Report," arXiv preprint arXiv:2505.09388, May 2025. Available: <https://arxiv.org/abs/2505.09388>

[2] Ollama, Inc., "Ollama, Get up and running with large language models," GitHub, 2023. Available: <https://github.com/ollama/ollama>

[3] Duffel, "Duffel Flights API Documentation," Duffel, 2024. Available: <https://duffel.com/docs/api/flights>

[4] Duffel, "Duffel Stays API Documentation," Duffel, 2024. Available: <https://duffel.com/docs/api/stays>

[6] Google LLC, "Angular, The web development framework for building the future," Google, 2025. Available: <https://angular.dev>

[7] OpenJS Foundation, "Express, Fast, unopinionated, minimalist web framework for Node.js," 2024. Available: <https://expressjs.com>

[8] D. Richard Hipp, "SQLite," SQLite Consortium, 2024. Available: <https://www.sqlite.org>

[9] Zetetic LLC, "SQLCipher, Open-source full-database encryption for SQLite," 2024. Available: <https://www.zetetic.net/sqlcipher>

[10] Duffel, "Duffel Cars API Documentation," Duffel, 2024. Available: <https://duffel.com/docs/api/cars>

[11] Microsoft Corporation, "Playwright, Reliable end-to-end testing for modern web apps," 2024. Available: <https://playwright.dev>